06/03/2026

Migration d'une Base de Données Relationnelle

Réaliser par Houy Antoine
En groupe avec Isaac Souidi et Aymen Guermassi

Phase 1 : Analyse des Données Sources et Modélisation Relationnel

Les données utilisées dans ce projet proviennent d'un fichier Excel contenant des statistiques officielles relatives aux crimes et délits enregistrés par les services de police et de gendarmerie en France.

Le fichier source est constitué de 21 feuilles :

- 1 feuille de présentation, qui décrit le contenu et la structure du fichier ;
- 10 feuilles correspondant aux données de la Police Nationale (PN) ;
- 10 feuilles correspondant aux données de la Gendarmerie Nationale (GN).

Chaque feuille correspond à une année d'observation. Les données couvrent donc plusieurs années et permettent d'analyser l'évolution des infractions enregistrées par les services de sécurité.

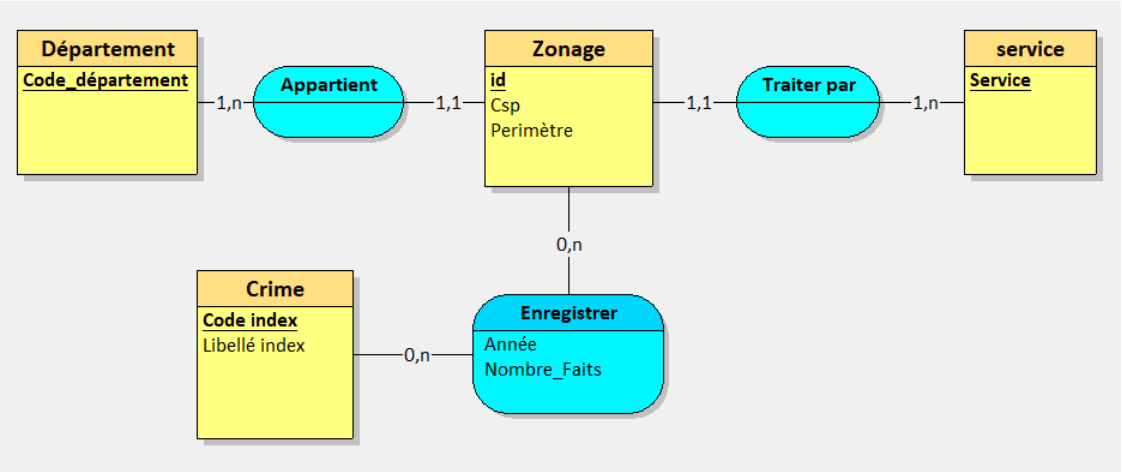
Dans chaque feuille, les informations sont organisées sous forme de tableau et contiennent notamment :

- le code de l'infraction (code index) ;
- le libellé de l'infraction ;
- les zones d'intervention (CSP ou unités territoriales) ;
- le périmètre géographique ;
- le département concerné ;
- le nombre de faits enregistrés.

La structure des feuilles diffère légèrement entre les données de la Police Nationale et celles de la Gendarmerie Nationale.

Nous avons commencé le travail par exporté au format CSV afin de faciliter leur traitement et leur intégration dans une base de données relationnelle.

À partir de l'analyse des données sources, un Modèle Conceptuel de Données (MCD) a été conçu afin d'identifier les principales entités et les relations entre elles.



Capture d'écran de notre MCD

Juste après cela nous avons récupérer le code sql donner par looping pour notre mcd, cela nous a permis après quelque modification de créer notre base sur db_browser.

```sql
CREATE TABLE Departement(
    code_departement VARCHAR(3),
    PRIMARY KEY(code_departement)
);

CREATE TABLE Crime(
    code_index INTEGER PRIMARY KEY AUTOINCREMENT,
    libelle_index VARCHAR(255)
);

CREATE TABLE Service(
    service VARCHAR(50),
    PRIMARY KEY(service)
);

CREATE TABLE Zonage(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    csp VARCHAR(100),
    perimetre VARCHAR(100),
    code_departement VARCHAR(3) NOT NULL,
    service VARCHAR(50) NOT NULL,
    FOREIGN KEY(code_departement) REFERENCES Departement(code_departement),
    FOREIGN KEY(service) REFERENCES Service(service)
);

CREATE TABLE Enregistrer(
    id INTEGER,
    code_index INTEGER,
    annee INTEGER,
    nombre_faits INTEGER,
    PRIMARY KEY(id, code_index, annee),
    FOREIGN KEY(id) REFERENCES Zonage(id),
    FOREIGN KEY(code_index) REFERENCES Crime(code_index)
);
```

Capture de la création de nos tables

J'ai importé les données dans la base de données à l'aide d'un script Python, deux problèmes principaux ont été identifiés après l'importation (voir importation db dans github).

Le premier problème provenait de la création de la table Zonage. En effet, la structure initiale de la table permettait l'insertion de plusieurs lignes identiques pour un même couple CSP et département, ce qui entraînait un grand nombre de doublons dans la base de données. Afin de corriger ce problème, il a été nécessaire de supprimer les doublons existants puis d'ajouter une contrainte d'unicité afin d'empêcher l'insertion de nouvelles lignes identiques.

```sql
DELETE FROM Zonage
WHERE id NOT IN (
    SELECT MIN(id)
    FROM Zonage
    GROUP BY csp, code_departement
);

CREATE UNIQUE INDEX zonage_unique
ON Zonage (csp, code_departement);
```

Le second problème concernait l'encodage des libellés dans la table Crime. Lors de l'importation des données, certains caractères accentués apparaissaient de manière incorrecte. Ce problème provenait d'un mauvais encodage des fichiers source lors de leur lecture par le script Python.

```sql
UPDATE Crime
SET libelle_index = CASE code_index

WHEN 1 THEN 'Règlements de compte entre malfaiteurs'
WHEN 2 THEN 'Homicides pour voler et à l''occasion de vols'
WHEN 3 THEN 'Homicides pour d''autres motifs'
WHEN 4 THEN 'Tentatives d''homicides pour voler et à l''occasion de vols'
WHEN 5 THEN 'Tentatives homicides pour d''autres motifs'
WHEN 6 THEN 'Coups et blessures volontaires suivis de mort'
WHEN 7 THEN 'Autres coups et blessures volontaires criminels ou correctionnels'
WHEN 8 THEN 'Prises d''otages à l''occasion de vols'
WHEN 9 THEN 'Prises d''otages dans un autre but'
WHEN 10 THEN 'Séquestrations'

ELSE libelle_index
END;
```

Conclusion

Ces deux problèmes étant corrigés à l'aide de différentes requêtes SQL, nos données son prête est complètement importé. Nous pouvons donc passer à la suite et nous diriger vers une migration de données vers Neo4j.

Phase 2 : Analyse des Limites du modèle relationnel

Après l'intégration des données dans une base de donnée relationnelle, j'ai pu observer plusieurs limites du modèle relationnel.

Tout d'abord, l'analyse des données nécessite souvent de relier plusieurs tables entre elles, notamment les tables *Crime*, *Zonage*, *Departement*, *Service* et *Enregistrer*, à l'aide de jointures. Cela peut rendre les requêtes plus complexes à écrire et plus difficiles à comprendre, avec un impact potentiel sur les performances.

Les bases de données relationnelles sont particulièrement efficaces pour gérer des données structurées, mais elles deviennent parfois moins adaptées lorsqu'il s'agit d'explorer de nombreuses relations entre les données.

Un autre point concerne la lisibilité des relations entre les différentes entités. Dans un modèle relationnel, les liens entre les tables sont représentés à l'aide de clés étrangères, ce qui rend parfois plus difficile la visualisation directe des relations entre les données. Il devient alors moins intuitif d'identifier rapidement les connexions existantes entre les différentes informations présentes dans la base de données.

Ces limitations m'ont amené à réfléchir à l'utilisation d'un modèle de base de données orienté graphe, qui permettrait de représenter plus naturellement les relations entre les entités.

Nous nous sommes donc tournés vers un modèle de graphe afin que l'on puisse représenter et d'exploiter de manière plus facile les données, et les nombreuses relations au sein de notre modèle.

Définition des nœuds

Dans le modèle graphe que je propose, plusieurs types de nœuds sont définis afin de représenter les principales entités présentes dans les données :

- Departement : représente un département français dans lequel les infractions sont enregistrées.
- Crime : représente un type d'infraction identifié par un code et un libellé.
- Service : représente l'organisme responsable de l'enregistrement des infractions, c'est-à-dire la Police nationale ou la Gendarmerie nationale.
- Zonage : représente une zone d'intervention (CSP ou unité territoriale) associée à un département et à un service.

Nous avons donc ici des nœuds qui contiennent différentes propriétés correspondant aux attributs présents dans la base relationnelle, par exemple le code du département ou le libellé d'un crime.

Définition des rélations

Les relations sont là pour montrer les liens entre les différents nœuds que nous avons dans notre graphe.

Dans ce modèle, plusieurs relations ont été définies :

- LOCALISE : relie un nœud *Zonage* à un nœud *Departement*. Quel département se situe une zone d'intervention.
- GERE : relie un nœud *Service* à un nœud *Zonage*. Quel service est responsable de la zone.

- ENREGISTRE : relie un nœud *Zonage* à un nœud *Crime*. Cette relation stock en elle des propriétés supplémentaires, l'année et le nombre de faits enregistrés.

Grâce à cette modélisation, il devient plus simple de représenter et d'explorer les relations entre les infractions, les territoires et les services de sécurité. Le modèle graphe permet également de réaliser plus facilement certaines analyses, comme l'évolution des crimes par territoire ou l'identification des infractions les plus fréquentes dans un département.

Phase 3 : Migration des Données vers un Modèle Graphe

Rappelle des objectifs de la migration

Cette migration va nous permettre de représenter les données sous forme de nœuds et de relations, ce qui nous facilitera l'exploration des connexions entre les différentes entités. Dans notre cas, les relations entre les crimes, les zones, les départements et les services peuvent être représentées de manière plus naturelle dans un graphe que dans un modèle relationnel comme évoqué précédemment.

Pour réaliser cette migration, nous allons donc développer un script Python. Ce script nous permettra de lire les données présentes dans la base SQLite et de les insérer automatiquement dans Neo4j en respectant la structure du modèle graphe défini précédemment.

Fonctionnement du script de migration

Que fait le script python ?

Le script Python commence par établir une connexion à la base de données SQLite, qui contient les tables créées lors des phases précédentes du projet. Cette connexion permet de lire les données présentes dans les tables *Departement*, *Service*, *Crime*, *Zonage* et *Enregistrer*.

Dans un second temps, une connexion est établie vers Neo4j, afin de pouvoir créer les nœuds et les relations dans la base de données graphe.

Avant d'importer les nouvelles données, le script commence par vider la base Neo4j afin d'éviter la présence d'anciennes données ou de doublons. Cette opération supprime tous les nœuds et toutes les relations précédemment stockées dans la base, ou bien des potentielles tentatives qui ne se serait pas terminée correctement.

Le script procède à l'importation des différentes entités :

```
Connexion OK
Suppression des anciennes données Neo4j
Base Neo4j vidée
Import Departement
Import Service
Import Crime
Import Zonage
```

- les départements sont convertis en nœuds *Departement* ;
- les services deviennent des nœuds *Service* ;
- les crimes sont transformés en nœuds *Crime* contenant le code et le libellé de l'infraction ;
- les zones d'intervention (Zonage) deviennent des nœuds *Zonage* contenant les informations relatives au CSP et au périmètre.

Une fois les nœuds créés, le script établit les relations entre les différentes entités :

- une relation LOCALISE relie chaque zone à son département ;
- une relation GERE relie chaque service à la zone dont il est responsable ;
- une relation ENREGISTRE relie une zone à un type de crime, avec des propriétés supplémentaires indiquant l'année et le nombre de faits enregistrés.

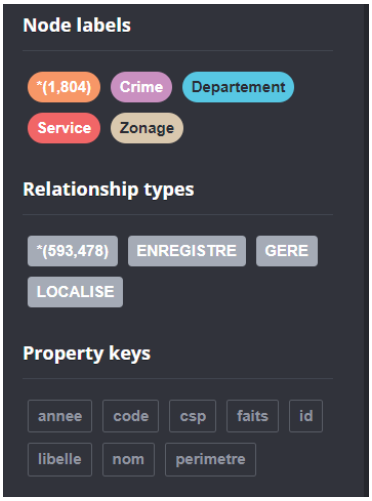
Le script importe uniquement les enregistrements pour lesquels le nombre de faits est supérieur à zéro afin d'éviter la création de relations inutiles dans le graphe et de surcharger notre modèle.

Enfin, une fois toutes les données importées, les connexions aux bases de données SQLite et Neo4j sont fermées, ce qui marque la fin du processus de migration. Pour suivre le processus, le code envoie une petite notification toutes les 5 000 relations créer (voir Migration dans github).

```
Relations créées : 560000
Relations créées : 565000
Relations créées : 570000
Relations créées : 575000
Relations créées : 580000
Relations créées : 585000
Relations créées : 590000
Migration terminée
```

Capture d'écran terminal script migration

Après cela, je retourne sur Neo4j et j'accède à ma base.



Capture d'écran Neo4j

La structure obtenue pour moi est celle permettant de représenter le plus clairement toutes les relations entre les données et nous allons donc pouvoir commencer les analyses et les tests sur notre base Neo4j directement maintenant.

Phase 4 : Validation et Exploitation de la Solution Graphe

Vérification des données

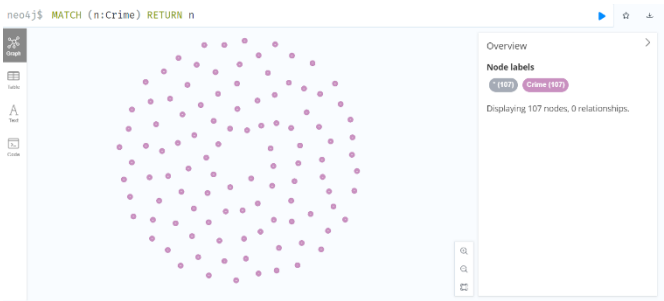
Cependant, avant cela, nous allons effectuer quelques tests de base afin de savoir si tous nos éléments se sont bien importés pour cela, je regarde les nombres d'éléments de ma table crime et zonage et je les compare avec mes tables sur db browser.

Table : Crime

	code_index	
	Filtre	Filtre
1	1	Règlements de comp
2	2	Homicides pour vol
3	3	Homicides pour d'a
4	4	Tentatives d'homic
5	5	Tentatives homicid
6	6	Coups et blessures
7	7	Autres coups et bl
8	8	Prises d'otages à
9	9	Prises d'otages da
10	10	Sequestrations
11	11	Menaces ou chantag
12	12	Menaces ou chantag
13	13	Atteintes à la dig
14	14	Violations de domi
15	15	Vols à main armée
16	16	Vols à main armée
17	17	Vols à main armée
18	18	Vols à main armée
19	19	Autres vols à main
20	20	Vols avec armes bl
21	21	Vols avec armes bl
22	22	Autres vols avec a
23	23	Vols violents sans

1 - 24 of 107

on voit ici que nous avons à la fois sur Neo4j et sur db_browse 107 ligne pour crime.



Si on regarde notre nombre de faits total :

Sur Neo4j

```
neo4j$ MATCH ()-[r:ENREGISTRE]→() RETURN SUM(r.faits) AS total_faits
```

	total_faits
1	35696578

Sur Db_browser

```
1  SELECT sum(nombre_faits) as fais
2  FROM Enregistrer
3
4
```

	fais
1	35696578

On se retrouve ici avec les mêmes chiffres ce qui appuie sur le fait que nos données se sont bien migrer.

Petit test de comparaison entre Neo4j et db_browser

```
1 MATCH (d:Departement)←[:LOCALISE]-(z:Zonage)-[r:ENREGISTRE]→(c:Crime)
2 RETURN d.code AS departement,
3        c.libelle AS crime,
4        SUM(r.faits) AS total_faits
5 ORDER BY total_faits DESC
```

	departement	crime	total_faits
1	"75"	"Autres vols simples contre des particuliers dans des locaux ou lieux publics"	561778
2	"75"	"Vols à la tire"	262950
3	"69"	"Autres vols simples contre des particuliers dans des locaux ou lieux publics"	154647

```sql
SELECT
    d.code_departement,
    c.libelle_index,
    SUM(e.nombre_faits) AS total_faits
FROM Enregistrer e
JOIN Crime c ON e.code_index = c.code_index
JOIN Zonage z ON e.id = z.id
JOIN Departement d ON z.code_departement = d.code_departement
GROUP BY d.code_departement, c.libelle_index
ORDER BY total_faits DESC;
```

	code_departement	libelle_index	total_faits
1	75	Autres vols simples contre des …	561778
2	75	Vols à la tire	262950
3	69	Autres vols simples contre des …	154647

On voit ici que pour la même requête, elle est bien plus énergivore et complexe en SQL. C'est donc pour ça qu'utiliser un modèle en graphe sur Neo4j est plus utile et pratique déjà pour un jeu de données comme celui-là. Dans le cas où l'on aurait un jeu de données avec encore plus de table en SQL, il serait en fait bien plus facile de traiter cela sûr avec du Nosql.

Les différentes méthodes de migrations qui existent

Lors de cette SAÉ et dans ce rapport, j'ai réalisé une migration de ma base de données SQL vers une base Neo4j à l'aide d'un script Python qui automatise l'ensemble du processus. Ce script se charge de lire les données présentes dans la base SQL, puis de créer automatiquement les nœuds et les relations dans Neo4j.

Cependant, il existe également à ma connaissance au moins 2 autres méthodes pour importer des données dans Neo4j.

La première méthode consiste à exporter chaque table de la base SQL sous forme de fichiers CSV. Ces fichiers peuvent ensuite être importés dans Neo4j, puis des requêtes Cypher permettent de recréer les relations entre les différentes entités.

Une autre méthode consiste à regrouper toutes les informations nécessaires dans un seul fichier CSV, préparé au préalable à l'aide d'un script Python. Ce fichier contiendrait déjà toutes les données correctement structurées. Il suffirait alors de l'importer directement dans Neo4j, par exemple en utilisant la fonctionnalité le drag and drop, puis de créer les relations à partir de ce fichier.

Ces différentes méthodes montrent qu'il existe plusieurs façons de réaliser une migration vers Neo4j, selon notre bon vouloir et la situation dans laquelle on se trouve.

Ajout données futures

Grâce à cette ligne dans mon fichier d'importation, même si on rajoute les csv 2022 il suffirait de les récupérer et de les mettre au même endroit que les csv des années précédente. Relancer le script importation db.

fichiers = [f for f in os.listdir(dossier_source) if f.lower().endswith(".csv")]

Puis par la suite, il suffira de relancer le code migration et dans notre Neo4j l'année 2022 serait importer.

Lien Github : https://github.com/Utopie81/Migration-de-donn-es-vers-ou-depuis-un-environnement-NoSQL